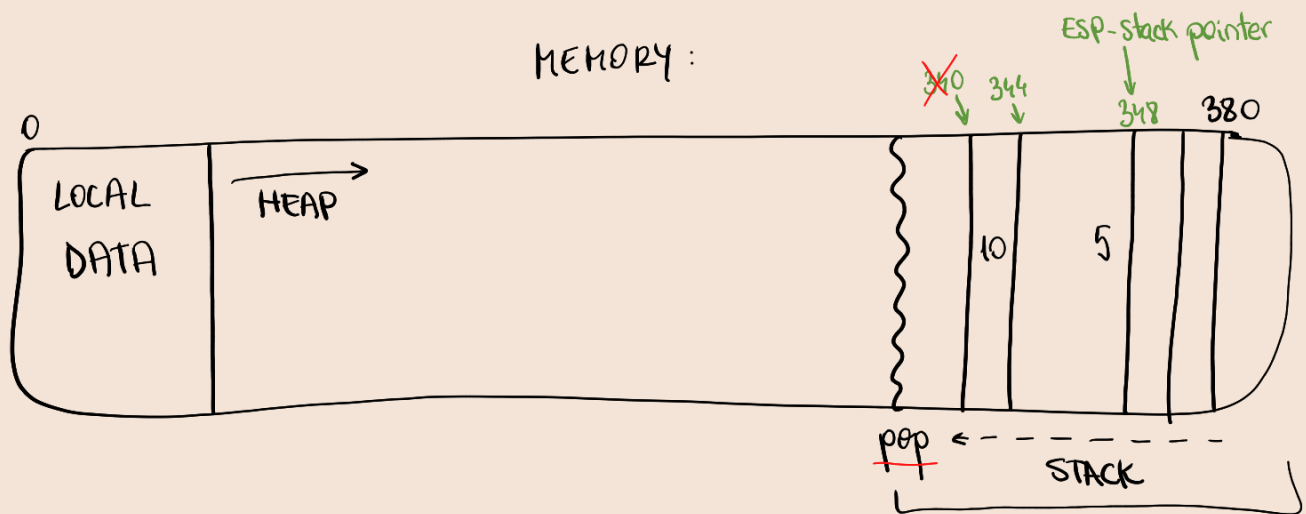


C Declaration Conventions (CDECL):

- 1) parameters on stack in reverse order ($\leftarrow$)
- 2) eax contains the return value (if any)
- 3) modifies registers (eax, ecx, edx, ...)
 - save registers → BEFORE function call pushad
 - AFTER function call popad → pushes on stack all registers.
- 4) the caller must free stack.

↓
add esp, 4 * nr_params



Ex: array of 100 nr → in local data.

dynamic allocated memory → heap (reuse with multiple purposes)
populate - clear - populate - clear...

push 5 { change esp with esp - 4 (1) sub ESP, 4 ($\leftarrow$)
create the slot (2) mov [ESP], dword 5.

↗ the one we want to extract.

pop eax { (1) mov eax, [ESP] (save value we popped)
(2) add esp, 4 (change the index →)

{ %d integer
 %s string

Segment data ...

format_scanf db "%d", 0 (=) scanf("%d", &a)

{
 push dword a
 push dword format_scanf
 call [scanf]
 add esp, 4*2

ASSEMBLY

bits 32

global start

extern exit, scanf, printf

import exit msvcrt.dll

import scanf msvcrt.dll

import printf msvcrt.dll

segment data use32 class = data:

 a dd 0

 b dd 0 !always dd

 s dd 0

 format_scanf db "%d", 0

 format_printf db "%d + %d = %d", 0

 msg_in1 db "Give a = ", 10, 0

 msg_in2 db "Give b = ", 10, 0

segment code use32 class = code:

start:

 push dword msg_in1

 call [printf]

 add esp, 4*1

 push dword a

 push dword format_scanf

 call [scanf]

 add esp, 4*2

C

#include <stdio.h>

int main()

{

 int a, b, sum;

 printf("Give a = \n");

 scanf("%d", &a);

 printf("Give b = \n");

 scanf("%d", &b);

 s = a + b

 printf("%d + %d = %d", a, b, s);

 return 0;

}

a = 5

b = 7

5 + 7 = 12

10, 13, 0

```

push dword msg-10h
call [printf]
add esp, 4*1

push dword b
push dword format_scanf
call [scanf]
add esp, 4*2

mov eax, [a]
add eax, [b]
push dword eax
push dword [b]
push dword [a]

push dword format_printf
call [printf]
add esp, 4*4
push dword 0
call [exit]

```

② Write a program that reads the content of a text file (a.txt), adds 1 to each byte and writes these bytes to a new file (b.txt). Rename b.txt to a.txt (after deleting a.txt).

ASSEMBLY

```

bits 32
global start
extern exit, ....

```

segment data...

```

filename1 db "a.txt", 0
filename2 db "b.txt", 0
mode_read db "r", 0
mode_write db "w", 0
c db 0
fd_in dd -1

```

C

```

int main()
{
    FILE *fd_in, *fd_out;
    char c;

    fd_in = fopen("a.txt", "r");
    if (fd_in > 0)
    {
        fd_out = fopen("b.txt", "w");
        if (fd_out > 0)
        {
            do
            {
                int count = fread(&c, sizeof(char), 1, fd_in);
                c = c + 1;
            }
        }
    }
}

```

```
fd_in dd 0
fd_out dd -1
msg db "error", 0
```

Segment code...

Start:

```
push dword mode_read
push dword filename1
call [fopen]
add esp, 4*2
cmp eax, 0
je else1
mov [fd_in], eax
push dword mode_write
push dword filename2
call [fopen]
add esp, 4*2
cmp eax, 0
je else2
mov [fd_out], eax
```

repeat:

```
push dword [fd_in]
push dword 1 <- 1 char
push dword 1 <- 1 byte once
push dword c
cmp eax, 0
je end
jmp repeat
call [fread]
add esp, 4*4
add byte [c], 1
push dword [fd_out]
push dword 1
push dword 1
push dword c
call [fwrite]
```

```
fwrite (&c, 1, 1, fd_out)
```

```
} while (count > 0);
```

```
}
```

```
else perror ("Error open output file")
```

```
fclose (fd_in)
```

```
}
```

```
else perror ("Error open input file")
```

```
exit(0);
```

```
}
```

add esp, 4*4

end:

```
push dword [fd-out]
call [fclose]
add esp, 4*1
```

else2:

```
push msg
call [perror]
add esp, 4
push dword [fd-in]
call [fclose]
add esp, 4*1
```

else1: ; we could not open the file.

```
push msg
call [perror]
add esp, 4
```

How to implement rename:

```
remove("a.txt")
rename("b.txt", "a.txt")
exit(0);
:
```

